Asynchronous QDI Circuit Synthesis from Signal Transition Protocols*

Bo-Yuan Huang[1], Yi-Hsiang Lai[2], Jie-Hong R.Jiang[1,2]
[1]Department of Electrical Engineering, National Taiwan University, Taipei, 10617, Taiwan
[2]Graduate Institute of Electronics Engineering, National Taiwan University, Taipei, 10617, Taiwan

ABSTRACT

Asynchronous circuits are promising in resolving the emerging issue of process variation and high synchronization power consumption. Among various asynchronous delay models, quasi-delay insensitive (QDI) model is the most robust and yet practical one due to its relaxed timing assumption. However, automatic synthesis of QDI circuits from signal transition graph (STG) protocol specification has not yet been proposed, despite the fact that algorithms synthesizing circuits under other delay models do exist. In this paper we propose the first algorithm synthesizing protocols specified in STGs into QDI circuits by analyzing STG structures without utilizing state graph assignment techniques. Furthermore, an optimization technique is proposed to simplify QDI circuits. In our synthesis algorithm, the state explosion issue is avoided, and restrictions on STGs are relaxed. Case studies on Advanced Microcontroller Bus Architecture (AMBA) and other protocols indicate the feasibility of our method.

1. INTRODUCTION

Asynchronous techniques gain their application in modern integrated circuit design. The rapid growth of system complexity makes synchronizing all flip flops in a chip difficult, and the variability of integrated circuit manufacturing makes the design-for-the-worst-case methodology inadequate. Synchronizing clock signals and controlling gate and interconnect delays within a chip can lead to substantial power consumption. Due to the promising strengths of asynchronous circuits in addressing these issues, myriad research efforts have been contributed; see, e.g., [1] for a brief history of asynchronous design.

Several asynchronous delay models have been proposed that vary in their underlying timing assumptions, such as delay insensitive (DI), quasi-delay insensitive (QDI), speed independent (SI), and other models. Among these delay models, the QDI model is the most robust and yet practical one. While the SI model assumes unbounded gate delay and zero wire delay, the QDI model has almost no timing assumption except for the *isochronic fork* assumption. QDI circuits assume the difference in time at which a signal arrives at each end of an isochronic fork is less than the minimum gate delay. This assumption can be easily met if the designated wires are physically localized in circuits. Due to their robustness, QDI circuits can be designed and synthesized in a way similar to the synchronous design flow [2–5].

Algorithms for control protocol synthesis are essential due to the thriving needs for system-level circuit designs. While signal transition graph (STG) is regarded as a specification formalism for asynchronous control protocols [6], many researchers studied asynchronous circuit synthesis from STGs. Also there are asynchronous synthesis tools utilize STGs to improve their performance [7]. Among the proposed STG synthesis algorithms, many solved the problem through state graph (SG) assignment, which requires generating intermediate SGs [6, 8, 9]. However, this approach would inevitably encounter the state explosion issue because the size of the SG is potentially exponential to the number of signals in the STG. Several methods such as STG decomposition [10,11], STG unfolding [11,12], and formulation with integer linear programming (ILP) [13] were proposed to tackle this issue. Other synthesis algorithms utilized graph-based approach to avoid constructing intermediate SGs [14–17]. Disregarding the state explosion issue, previous algorithms were only applicable on STGs with complete state coding (CSC) property. Furthermore, the property *persistency* is also required for SI circuits [18]. Several methodologies aiming for efficient verification/modification for CSC and persistency property were proposed [12, 19]. Apart from the extra cost for validity verification, the persistency property of STGs might reduce concurrency and degrade circuit performance due to its additionally imposed signal transition constraints. Control protocol synthesis from other specifications were also proposed [20].

In order to synthesize STGs into QDI circuits without the requirements of CSC and persistency property, we define five behavior models and propose a method to analyze the topological relation between signal transitions in *single-cycle* STGs. Every output transition and its enabling transitions can be completely classified into the five behavior models. We use *triggering condition* (TC) to describe the conditions for a signal to change its value concerning logic levels of input signals; *triggering pattern* (TP) for each behavior model is derived to compose TCs for signals in *simply-driven* STGs. Based on the TCs we derived, output signals synthesized are *effective*, *stable* and *precise* under the QDI assumption. We also propose optimization techniques to reduce circuit cost by restricting system concurrency. Overall synthesis algorithm has complexity $O(|E|^2)$, where $|E|$ is the number of edges of the underlying STG. Experimental results suggest that our method can be effective in synthesizing circuits from practical protocols, such as the AMBA specification.

2. PRELIMINARIES

A *Petri net* [21] is a tuple (Q, T, F, M_0), where Q is a set of *places*, T is a set of *transitions*, $F \subset (Q \times T) \cup (T \times Q)$ defines the relation flow among Q and T, and M_0 represents the initial marking. If $(t, q) \in F$ then $t \in T$ is an input transition of $q \in Q$; if $(q, t) \in F$ then $t \in T$ is an output transition of $q \in Q$. A *marking* is a set of non-negative integers that represents the number of tokens on each place. A transition t is enabled and may be *fired* if all of its input places have at least one token. We also call a transition is *triggered* or occurs if it is *fired*. Note that a transition may not be immediately triggered even if it is enabled. We denote $Reach(M_0)$ as the set of reachable markings given initial marking M_0.

A *path* P is an ordered set of transitions that represents a part of firing sequence. If transition u^* is fired within path P, we denote it as $u^* \in P$. A *cycle* is a path with the same starting and ending transition. A path or cycle is called *simple* if all transitions in it appears exactly once.

In this paper, we focus on a special Petri net, called a *marked*

*This work was supported in part by the Ministry of Science and Technology of Taiwan under grants MOST 103-2221-E-002-273, 103-2622-E-002-031, and 104-2628-E-002-013-MY3.

graph, where every place has exactly one input transition and one output transition. A marked graph is *live* and *safe* if it is strongly connected and every place contains no more than one token within any marking reachable from the initial marking [16]. Liveness of STGs guarantees no deadlock occurs [18]. In addition, a marked graph is *simple* if its initial marking M_0 places exactly one token on each simple cycle.

Definition 1. *A* signal transition graph *(STG)* $G = (V, E)$ *is an interpreted subset of marked graph wherein each transition represents either the rising* (x^+) *or falling* (x^-) *of a signal* x*, which has signal levels* high *and* low*.* V *is the set of transitions and* E *is the set of edges corresponding to places of the underlying marked graph.*

We denote the set of signals as S, the set of input signals as PI and the set of output signals as PO, that is to say, $S = PI \cup PO$ and $PI \cap PO = \emptyset$. We define x^* as an arbitrary transition of signal x, which is either x^+ or x^-. We call an STG *valid* if its underlying marked graph is *simple*, *live* and *safe*. Taking T-element [22] in Figure 1(a) for instance, $PI = \{lr, Oa\}$, $PO = \{la, Or\}$ and $S = \{lr, la, Or, Oa\}$. We label transitions of input and output signals in red and black, respectively, in all the following figures.

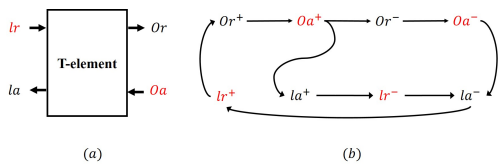

Figure 1: T-element (a) IO block (b) STG

Definition 2. *A signal is called* single-cycle *if there is exactly one rising transition and one falling transition within the STG. Otherwise, it is called a multi-cycle signal. A* single-cycle *STG contains only single-cycle signals.*

In this paper We focus on single-cycle STGs to study the relation between STG structure and signal transitions. In a single-cycle STG, the rising transition x^+ and falling transition x^- of a signal x are designated as the *conjugate transition* of each other, and is denoted by $\neg$, e.g., $x^{\neg+} = x^-$.

Definition 3. *Two transitions* x^**,* y^* *are* concurrent*, denoted as* $(x^* \parallel y^*)$*, if the two transitions are both enabled within some markings reachable from the initial marking* M_0*. In contrast, two transitions* x^**,* y^* *are* sequential*, denoted as* $(x^* \sim y^*)$*, if the two transitions are not enabled simultaneously within any marking reachable from the initial marking* M_0*.*

If two transitions are sequential, triggering order between them can be defined given a reference transition. We write $(u^* \gg x^* \gg y^*)$ to denote the cases where transition x^* must be triggered before transition y^* fires once transition u^* has been triggered. For instance, in Figure 1(b), $(Or^+ \sim Oa^-)$, $(la^+ \parallel Oa^-)$, and $(lr^+ \gg Oa^+ \gg la^-)$.

We denote $\mathbf{P_s}(x^*, y^*)$ as a set of simple paths that start from x^* and end with y^*, i.e., $\mathbf{P_s}(x^*, y^*) = \{P \mid P = \{x^*, \cdots, y^*\}\}$. We define the symbol $\in_\diamond$ as $u^* \in_\diamond \mathbf{P_s}(x^*, y^*) \Leftrightarrow \exists P \in \mathbf{P_s}(x^*, y^*)$ s.t. $u^* \in P$. For example, there exists a simple path $P = \{Or^+, Oa^+, la^+, lr^-, la^-\}$ in Figure 1(b) and $P \in \mathbf{P_s}(Or^+, la^-)$. Also, $Or^- \in_\diamond \mathbf{P_s}(Or^+, la^-)$ and $lr^+ \notin_\diamond \mathbf{P_s}(Or^+, la^-)$.

Lemma 1. *In a valid STG,* $(x^* \sim y^*) \Leftrightarrow \exists P_1 \in \mathbf{P_s}(x^*, y^*) \wedge \exists P_2 \in \mathbf{P_s}(y^*, x^*)$ *such that the cycle formed by* P_1 *and* P_2 *is simple.*

Proof. Within a simple, live and safe marked graph, two transitions are concurrent if and only if they do not reside on a simple cycle [6,16]. Since a valid STG is an interpreted simple, live and safe marked graph by Definition 1, the existence of the two paths are therefore implied. □

Lemma 2. *In a valid and single-cycle STG, every transition must be sequential to its conjugate transition, i.e.,* $\forall u \in S, (u^+ \sim u^-)$*.*

Proof. If there exists $u \in S$ with $(u^+ \parallel u^-)$, then there exists $M_i \in Reach(M_0)$ such that both u^+ and u^- are simultaneously enabled under M_i, which will cause ambiguity in signal level. □

The following corollary follows from the above two lemmas.

Corollary 1. *Given a valid and single-cycle STG, there exists a path connecting conjugate transitions of each signal, i.e.,* $\forall u \in S$*,* $\exists P_1 \in \mathbf{P_s}(u^+, u^-) \wedge \exists P_2 \in \mathbf{P_s}(u^-, u^+)$*.*

We define the *enabling set* $En(t^*)$ of a transition t^* to be the set of input transitions, i.e., $En(t^*) = \{u^* \mid (u^*, t^*) \in E\}$. Taking T-element in Figure 1(b) for example, $En(la^+) = \{Oa^+\}$ and $En(la^-) = \{lr^-, Oa^-\}$.

2.1 STG Properties and Handshake Protocols

We focus on STGs containing only input and output signals. Because internal signals are not necessary in representing control protocols and, nevertheless, they can be transformed to dummy outputs or be eliminated.

Due to the characteristics of QDI circuits, we focus on valid STGs whose input transitions and output transitions interleave. The reasons are twofold. First, the order of consecutive output transitions is not very meaningful under the QDI model, where wire delays is unbounded and the environment may thus receive transitions in different order. Second, edges in an STG indicate the causality between transitions, thus input transitions should be triggered only by output transitions. Our assumption on IO-interleaving is reasonable for QDI circuits. Furthermore, with IO-interleaving, the synthesized circuits have no internal feedback, which may results in signal metastability.

Definition 4. *In a valid, single-cycle, and IO-interleaving STG, an input* x *is* simply-driven *if* $\forall u^* \in En(x^+), u^{\neg*} \in En(x^-)$*, and* $\forall v^* \in En(x^-), v^{\neg*} \in En(x^+)$*. An STG is* simply-driven *if it is a valid, single-cycle, and IO-interleaving STG with all input signals being simply-driven.*

For a simply-driven (SD) STG, input transitions change their signal levels with respect to certain output transitions. It is similar to the request-acknowledge handshake mechanism. Taking T-element in Figure 1(b) for instance, $En(lr^+) = \{la^-\}$ and $En(lr^-) = \{la^+\}$. Input signal lr changes its signal level depends only on the output it received, say signal la, and it rises and falls under complementary conditions. This paper focuses on *simply-driven* STGs. With our case study on AMBA protocol and empirical experience, we believe this assumption is practical for real-life protocols under QDI realization.

3. STG STRUCTURE ANALYSIS

In this section we propose a formal method to analyze relations between signal transitions and define five *behavior models* to represent topological relations between output transitions and their enabling transitions. In the following discussion, we assume

$$u^* \in En(x^*). \tag{1}$$

We study the cases whether $u^{\neg*}$ is concurrent or sequential to x^* or $x^{\neg*}$, and we categorize the topological orders of related transitions u^*, $u^{\neg*}$, x^*, and $x^{\neg*}$ into the *Staggered*, *Back-to-Back*, *Sneaking*, *Liable*, and *Independent* models. Each model will be characterized with two equivalent forms: *topological relations* and *path constraints*.

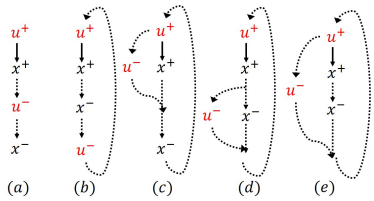

Figure 2: STG examples: (a) Staggered, (b) Back-to-Back, (c) Sneaking, (d) Liable, (e) Independent.

Figure 2 depicts five examples of topological relations of the five models under $u^+ \in En(x^+)$, where solid lines represent edges in the STG and dotted lines represent paths. Note that the STGs of Figure 2 only show essential transitions and edges; they do not mean to exclude other transitions and edges.

3.1 Staggered

The transitions u^* and x^* in this model are sequential with $u^{\neg *}$ after x^* and before $x^{\neg *}$. The topological relation of this model is

$$(u^{\neg *} \sim x^*) \wedge (u^{\neg *} \sim x^{\neg *}) \wedge (x^* \gg u^{\neg *} \gg x^{\neg *}).$$

The transitions u^* and x^* belong to the *Staggered* model if they satisfy the path constraint

$$u^{\neg *} \in_\diamond \mathbf{P_s}(x^*, x^{\neg *}) \wedge$$
$$(u^{\neg *} \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^*) \vee \forall P \in \mathbf{P_s}(x^*, x^{\neg *}), u^{\neg *} \in P).$$

3.2 Back-to-Back

Similar to the *Staggered* model, transitions u^* and x^* in this model are mutually sequential. However, transition $u^{\neg *}$ follows $x^{\neg *}$ once x^* is triggered. The topological relation of this model is

$$(u^{\neg *} \sim x^*) \wedge (u^{\neg *} \sim x^{\neg *}) \wedge (x^* \gg x^{\neg *} \gg u^{\neg *}).$$

The transitions u^* and x^* belong to the *Back-to-Back* model if they satisfy the path constraint

$$u^{\neg *} \in_\diamond \mathbf{P_s}(x^{\neg *}, x^*) \wedge$$
$$(u^{\neg *} \notin_\diamond \mathbf{P_s}(x^*, x^{\neg *}) \vee \exists P \in \mathbf{P_s}(x^*, x^{\neg *}), u^{\neg *} \notin P).$$

3.3 Sneaking

In this model only $x^{\neg *}$ is sequential to $u^{\neg *}$, and $u^{\neg *}$ fires before $x^{\neg *}$ once u^* has been triggered. The topological relation of this model is

$$(u^{\neg *} \parallel x^*) \wedge (u^{\neg *} \sim x^{\neg *}) \wedge (u^* \gg u^{\neg *} \gg x^{\neg *}).$$

The transitions u^* and x^* belong to the *Sneaking* model if they satisfy the path constraint

$$u^{\neg *} \notin_\diamond \mathbf{P_s}(x^*, x^{\neg *}) \wedge u^{\neg *} \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^*) \wedge$$
$$u^{\neg *} \in_\diamond \mathbf{P_s}(x^{\neg *}, x^{\neg *}) \wedge u^{\neg *} \notin_\diamond \mathbf{P_s}(x^*, x^*).$$

3.4 Liable

In this model only x^* is sequential to $u^{\neg *}$, and transition $u^{\neg *}$ must occurs after x^* once u^* has occurred. The topological relation of this model is

$$(u^{\neg *} \sim x^*) \wedge (u^{\neg *} \parallel x^{\neg *}) \wedge (u^* \gg x^* \gg u^{\neg *}).$$

Transitions u^* and x^* belong to the *Liable* model if they satisfy the path constraint

$$u^{\neg *} \notin_\diamond \mathbf{P_s}(x^*, x^{\neg *}) \wedge u^{\neg *} \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^*) \wedge$$
$$u^{\neg *} \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^{\neg *}) \wedge u^{\neg *} \in_\diamond \mathbf{P_s}(x^*, x^*).$$

3.5 Independent

In this model transition $u^{\neg *}$ is sequential to neither x^* nor $x^{\neg *}$. The topological relation of this model is

$$(u^{\neg *} \parallel x^*) \wedge (u^{\neg *} \parallel x^{\neg *}).$$

Transitions u^* and x^* belong to the *Independent* model if they satisfy the path constraint

$$u^{\neg *} \notin_\diamond \mathbf{P_s}(x^*, x^{\neg *}) \wedge u^{\neg *} \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^*) \wedge$$
$$u^{\neg *} \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^{\neg *}) \wedge u^{\neg *} \notin_\diamond \mathbf{P_s}(x^*, x^*)$$

With the above categorization, analysis on topological structure can be applied to every transition and its enabling set. Furthermore, either two transitions are sequential or concurrent can be determined by the existence of path between them, which can be efficiently determined by a depth-first traversal.

Lemma 3. *For valid and single-cycle STGs, the topological relation and the path constraint characterized by each of the above five models are equivalent.*

Proof. The equivalences between the topological relation and the path constraint under models *Sneaking*, *Liable* and *Independent* follow from Lemma 1.

For models *Staggered* and *Back-to-Back*, we consider the following two conditions:

$$u^{\neg *} \in_\diamond \mathbf{P_s}(x^*, x^{\neg *}) \quad (2)$$
$$u^{\neg *} \in_\diamond \mathbf{P_s}(x^{\neg *}, x^*) \quad (3)$$

In both *Staggered* and *Back-to-Back* models, at least one of the conditions (2) and (3) holds due to $(u^{\neg *} \sim x^*) \wedge (u^{\neg *} \sim x^{\neg *})$, implied by Lemma 1. Transitions that satisfy (2) but not (3) can be clearly classified by the *Staggered* model. Similarly, transitions that satisfy (3) but not (2) can be classified by the *Back-to-Back* model.

For transitions that satisfy both (2) and (3), there are several cases: If $\forall P \in \mathbf{P_s}(x^*, x^{\neg *})$, $u^{\neg *} \in P$, then the transitions should certainly be classified with the *Staggered* model since $x^{\neg *}$ would never be enabled before $u^{\neg *}$ is fired. If $\exists P \in \mathbf{P_s}(x^*, x^{\neg *})$ such that $u^{\neg *} \notin P$, the transitions may be enabled in different orders depending on the initial markings as the examples shown in Figure 3, where blue circles indicate initial markings. Note that the differences between Figure 3 (a) and (b) and between (c) and (d) are whether $u^* \in_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$.

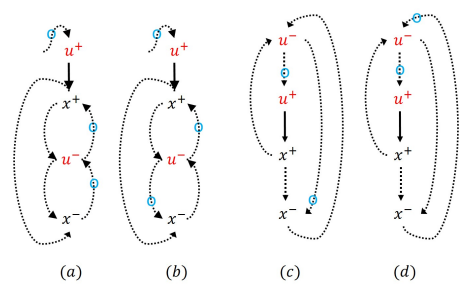

Figure 3: STG examples for Staggered and Back-to-Back model classification.

Transitions where tokens for $u^{\neg *}$ are ready before those for $x^{\neg *}$ once u^* and x^* are fired, are classified the *Staggered* model; otherwise they are classified by the *Back-to-Back* model. The former, however, must yield more than one token on some simple path $P \in \mathbf{P_s}(x^{\neg *}, x^*)$ with $u^{\neg *} \in P$, and thus violates the STG validity defined in Definition 1. Examples are shown in Figure 3(a) and (d). Only those initial markings, as shown in Figure 3(b) and (c), resulting in *Back-to-Back* model may be valid. As a consequence, transitions satisfying both (2) and (3) while $\exists P \in \mathbf{P_s}(x^*, x^{\neg *})$ such that $u^{\neg *} \notin P$ should be classified by the *Back-to-Back* model.

Therefore, for a valid and single-cycle STG, the topological relation and the path constraint characterized by each of the behavior model are equivalent. □

THEOREM 1. *Given any valid and single-cycle STG, the above five behavior models completely cover all possible structures among transitions.*

PROOF. Based on Lemma 3, the possible structures among transitions that are covered by the five models can be characterized by the union of the path constraints defined by the five models. Taking the union of the path constraints of the *Sneaking*, *Liable*, and *Independent* models, we have

$$u^{\neg *} \notin_{\diamond} \mathbf{P_s}(x^*, x^{\neg *}) \wedge u^{\neg *} \notin_{\diamond} \mathbf{P_s}(x^{\neg *}, x^*) \wedge (u^{\neg *} \notin_{\diamond} \mathbf{P_s}(x^{\neg *}, x^{\neg *}) \vee u^{\neg *} \notin_{\diamond} \mathbf{P_s}(x^*, x^*)). \quad (4)$$

On the other hand, taking the union of the path constraints of the *Staggered* and *Back-to-back* models, we have

$$u^{\neg *} \in_{\diamond} \mathbf{P_s}(x^*, x^{\neg *}) \vee u^{\neg *} \in_{\diamond} \mathbf{P_s}(x^{\neg *}, x^*). \quad (5)$$

which is the union of Eq. (2) and Eq. (3). Hence, we only have to consider the rest possible cases, that is, both Eq. (2) and Eq. (3) fail and $u^{\neg *} \in_{\diamond} \mathbf{P_s}(x^*, x^*) \wedge u^{\neg *} \in_{\diamond} \mathbf{P_s}(x^{\neg *}, x^{\neg *})$. We show that such cases do not exist. The assumption $\exists P_1 \in \mathbf{P_s}(x^*, x^*)$ with $u^{\neg *} \in P_1$ and $\exists P_2 \in \mathbf{P_s}(x^{\neg *}, x^{\neg *})$ with $u^{\neg *} \in P_2$ may imply the existence of a path P so that $P \in \mathbf{P_s}(x^*, x^{\neg *}) \cup \mathbf{P_s}(x^{\neg *}, x^*)$, which contradicts to (2) and (3). Therefore such cases are excluded.

As a consequence, the transitions of a valid and single-cycle STG are completely covered by the five behavior models. □

4. TRIGGERING CONDITIONS AND TRIGGERING PATTERNS

To derive the conditions for output signals to change their signal levels, we define *triggering condition* (TC) to represent the conditions for a transition to be triggered, and derive *triggering pattern* (TP), which describes how an output signal is affected by other signals, to comprise TC for every output signal. We propose a direct synthesis method for every model guaranteeing that the circuit synthesized behaves the same as the original STG specified. We further propose optimization techniques for *Staggered*, *Sneaking*, and *Liable* models by restricting system concurrency to reduce final circuit size.

DEFINITION 5. *In representing* triggering condition *(TC), we denote $R_b(x)$ as the* pull-up set *and $F_b(x)$ as the* pull-down set *of signal x. The pull-up and pull-down sets of a signal are sets of transitions whose triggering are necessary for the signal to rise and fall, respectively.*

Taking the standard two-input C-element [23] for example, let the input signals be a, b and output signal x. Then the TC of x is $R_b(x) = \{a^+, b^+\}$ and $F_b(x) = \{a^-, b^-\}$. *Note that forks to the pull-up set and to the pull-down set of a same signal are designated isochronic.*

In order to derive TC for every output signal, we analyze the behavior model to which every output transition and its enabling transition belong. Based on the model derived, we gradually add elements into TCs by applying TPs, which are expressed in the format:

$$x^* = R(u^*)F(t^*). \quad (6)$$

It denotes, for some u^* and t^*, adding u^* to $R_b(x)$ and adding t^* to $F_b(x)$ if $x^* = x^+$, or adding u^* to $F_b(x^*)$ and t^* to R_b^* if $x^* = x^-$. For the C-element, the TC of output x in terms of TP is expressed as $x^+ = R(a^+)F(a^-)$ and $x^- = R(b^-)F(b^+)$.

A signal is called *effective* if it is enabled once every enabling transition has been fired; a signal is called *stable* if there is no glitch between rising and falling transition; a signal is called *precise* if it guarantees no ambiguity between rising and falling transitions.

PROPOSITION 1. *If, for some signal x, there exists a transition in pull-up set whose conjugate transition is contained in pull-down set, i.e., $\exists u^* \in V$ such that $u^* \in R_b(x) \wedge u^{\neg *} \in F_b(x)$, then the signal x is precise.*

Implementing an STG with all behaviors being preserved may be at the expense of circuit complexity. Since reduction on system flexibility may lead to improvement in circuit cost, we define the *conservative* property below to constrain circuit synthesis with system flexibility.

DEFINITION 6. *A valid STG G_1 is* conservative *to a valid STG G_2 if the system behavior, a set of signal transition traces, of G_1 is a subset of that of G_2.*

LEMMA 4. *Adding one edge on a valid STG guarantees it to be conservative to the original STG if no deadlock is caused.*

PROOF. Consider adding an edge $E = (Y, X)$ on a valid STG, where X and Y are transitions in the graph. The transition traces may differ, due to edge E, only when X is originally enabled but disabled after the edge E is added. There are two possible situations. First, X is disabled at some time and remains disabled forever, which is deadlock. The second situation is that X is disabled at some time and will be enabled after some finite transitions. For the latter, we claim that the delay in the enabling of transition X causes no difference in system behavior. As mentioned in the definition of STG in Section 2, a transition may, but not necessarily, be triggered if it is enabled. That is, a transition may be enabled but not triggered for any arbitrary time, which behaves the same as the delay of enabling. Therefore, if adding an edge does not cause deadlock, the new STG is conservative to the original STG. □

We elaborate the TP for each behavior model in the following subsections.

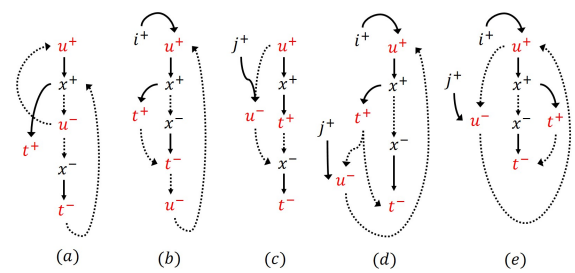

Figure 4: STG examples for direct synthesis: (a) Staggered, (b) Back-to-Back, (c) Sneaking, (d) Liable, and (e) Independent.

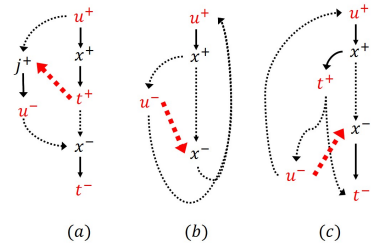

Figure 5: STG examples for optimization: (a) Sneaking, (b) Liable, and (c) Liable.

4.1 Staggered

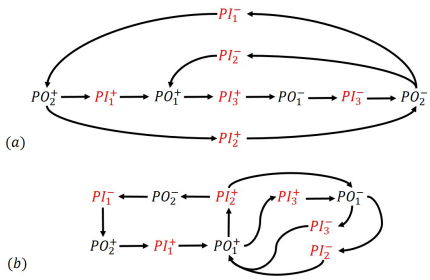

Figure 6: STG examples: (a) Back-to-Back, and (b) Liable.

4.1.1 Direct Synthesis

The triggering pattern for *Staggered* model depends on STG structure. If $u^* \in_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$, as the case of Or^- and Oa^+ in T-element, then TP is

$$x^* = R(u^*)F(u^{\neg *}).$$

On the other hand, if $u^* \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$, an input signal t should be designated such that $x^* \in En(t^*)$, $x^{\neg *} \in En(t^{\neg *})$ and $t^{\neg *} \in_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$. Such input signal t exists due to Corollary 1 and the SD property. An STG example is shown in Figure 4(a). The TP for *Staggered* model, if $u^* \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$, is

$$x^* = R(u^*, t^{\neg *})F(t*).$$

Taking the T-element of Figure 1 for example, x^*, u^* and t^* correspond to la^+, Oa^+ and lr^-, respectively.

LEMMA 5. *Given a simply-driven STG with $u^* \in En(x^*)$, u in PI, and x in PO, applying TP for* Staggered *model guarantees the underlying output signal x to be effective, stable, and precise with respect to signal u.*

PROOF. If $u^* \in_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$, then signal u must retain its signal level till transition x^* is to be triggered, and transition $u^{\neg *}$ will not occur before transition x^* is triggered. On the other hand, if $u^* \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$, then transitions u^* and $t^{\neg *}$ must have occurred before x^* should be triggered. Note that transition $x^{\neg *}$ behaves as completion detection of transition t^*. Hence signal x is precise due to Proposition 1. □

4.1.2 Optimization

In optimizing signals with transition belonging to *Staggered* model, we can apply simpler TP with

$$x^* = R(u^*).$$

LEMMA 6. *Given a simply-driven STG with $u^* \in En(x^*)$, u in PI, and x in PO, applying TP as $x^* = R(u^*)$ for* Staggered *model guarantees the underlying output signal x to be effective and stable with respect to signal u.*

PROOF. Signal u holds its signal level at the time when x^* is to be triggered because $u^{\neg *}$ will not be enabled before $x^{\neg *}$ has been triggered. □

Note that this optimization does not guarantee output signal x to be precise. Nevertheless, all TPs in *Back-to-Back*, *Sneaking*, *Liable* and *Independent* models suffice to make signal x precise as we will show in later sections. As a result, optimization for *Staggered* model can be applied if any of the enabling transition of signal x does not belong to *Staggered* model. Note that no additional edges are added in optimization for *Staggered* model.

4.2 Back-to-Back

To specify TP for *Back-to-Back* model, an input signal t and an output signal i should be designated. In addition, an internal signal B_{ux} is introduced in order to keep signal x stable. *The fork from input signal u to output signal x and to internal signal B_{ux} is designated isochronic.* Signal i can be any signal satisfying $i^* \in En(u^*)$, while signal t is chosen such that $x^* \in En(t^*)$, $x^{\neg *} \in En(t^{\neg *})$, and $t^{\neg *} \in_\diamond \mathbf{P_s}(x^{\neg *}, u^{\neg *})$. Such input signal t exists due to Lemma 1 and the SD property. Figure 4(b) shows an example. The TP of *Back-to-Back* model is specified by

$$\begin{aligned} B^*_{ux} &= R(u^*, t^*)F(u^{\neg *}), \\ x^* &= R(u^*, B^{\neg *}_{ux})F(B^*_{ux}), \\ i^* &= R(B^{\neg *}_{ux}), \end{aligned}$$

where B^*_{ux} has the same sign as x^*. Taking the STG of Figure 6(a) for example, x^*, u^*, t^*, and i^* correspond to PO_1^+, PI_1^+, PI_3^+, and PO_2^+, respectively.

LEMMA 7. *Given a simply-driven STG with $u^* \in En(x^*)$, u in PI, and x in PO, applying TP for* Back-to-Back *model guarantees the underlying output signal x to be effective, stable, and precise with respect to input signal u.*

PROOF. This proof is similar to the proof of Lemma 5. □

4.3 Sneaking

4.3.1 Direct Synthesis

To specify TP for *Sneaking* model, an input signal t and output signal j have to be designated. Signal j can be any signal if $j^* \in En(u^{\neg *})$. Signal t is chosen such that $x^* \in En(t^*)$, $x^{\neg *} \in En(t^{\neg *})$ and $t^* \in_\diamond \mathbf{P_s}(x^*, x^{\neg *})$. Such input signal t exists due to Lemma 2 and the SD property. We also create an internal signal B_{ux} to avoid deadlock by considering the completion detection of signal x. *The fork from input signal u to output signal x and to internal signal B_{ux} is designated isochronic.* An example is shown in Figure 4(c). The TP for *Sneaking* model is specified by

$$\begin{aligned} B^*_{ux} &= R(u^*)F(u^{\neg *}, t^*), \\ x^* &= R(B^*_{ux})F(B^{\neg *}_{ux}), \\ j^* &= R(B^*_{ux}), \end{aligned}$$

where B^*_{ux} has the same sign as x^*. Taking T-element in Figure 1 for example, x^*, u^*, t^*, and j^* correspond to la^+, Oa^+, lr^-, and Or^-, respectively.

LEMMA 8. *Given a simply-driven STG with $u^* \in En(x^*)$, u in PI, and x in PO, applying TP for* Sneaking *model guarantees the underlying output signal x to be effective, stable, and precise with respect to input signal u.*

PROOF. This proof is similar to the proof of Lemma 5. □

4.3.2 Optimization

To simplify TP for *Sneaking* model, we may search an output transition j^* such that $j^* \in En(u^{\neg *})$ and $u^{\neg *}$ is the only fanout transition of j^*. If such transition exists, we select an input transition t^* satisfying $x^* \in En(t^*)$ and x^* is the only fanin of t^*, as shown in Figure 5(a). The TP can be simplified as

$$\begin{aligned} x^* &= R(u^*)F(u^{\neg *}), \\ j^* &= R(t^*). \end{aligned}$$

Taking T-element in Figure 1 for example, x^*, u^*, t^*, and j^* correspond to la^+, Oa^+, lr^-, and Or^-, respectively.

This optimization simplify TP by adding an additional edge to the original STG, as the red dash line shows in Figure 5(a). System concurrency is reduced since transition $u^{\neg *}$ cannot occur until transition t^* completes, making it behaves similar to *Staggered* model. We select transition j^* and t^* with $u^{\neg *}$ and x^* being their only fanout, respectively, to avoid deadlock and minimize concurrency reduction. However, different criteria should be studied in the future.

LEMMA 9. *Given a simply-driven STG, the above optimization for* Sneaking *model guarantees the modified STG to be deadlock-free and conservative to the original STG.*

PROOF. In the *Sneaking* model, x^* and $u^{\neg *}$ are concurrent, which implies no simple cycle contains both the two transitions due to Lemma 1. Because we choose j^* and t^* with $u^{\neg *}$ and x^* being their only fanout, respectively, no simple cycle contains both j^* and t^*. Transition j^* and t^* are therefore concurrent. Because a simply-driven STG is deadlock-free and adding an edge between two *concurrent* transitions will not cause deadlock, the optimization approach guarantees the modified STG to be deadlock-free and conservative to the original one due to Lemma 4. □

4.4 Liable

4.4.1 Direct Synthesis

The TP for *Liable* model depends on the STG structure. If $x^* \in En(u^{\neg *})$, e.g., $PI_2^+ \in En(PO_2^-)$ in Figure 6(b), then the TP is

$$x^* = R(u^*)F(u^{\neg *}).$$

However, if $x^* \notin En(u^{\neg *})$, then an input signal t and output signals i and j should be designated. Signals i and j can be arbitrary signals as long as $i^* \in En(u^*)$ and $j^* \in En(u^{\neg *})$. Signal t is chosen such that $x^* \in En(t^*)$, $x^{\neg *} \in En(t^{\neg *})$, and $t^* \in_\diamond \mathbf{P_s}(x^*, u^{\neg *})$. Due to Lemma 1 and the SD property signal t with the above constraints exists. Under these criteria, signal t does not change its signal level until $x^{\neg *}$ occurs, which can be utilized as a substitute for $u^{\neg *}$. In addition, an internal signal B_{ux} is introduced to keep signal x stable. *The fork from input signal u to output signal x and to internal signal B_{ux} is designated isochronic.* Figure 4(d) shows an STG example. Taking T-element in Figure 1 as another example, x^*, u^*, t^*, i^*, and j^* correspond to Or^+, lr^+, Oa^+, la^-, and la^+, respectively. The TP for *Liable* model, if $x^* \notin En(u^{\neg *})$, is

$$\begin{aligned} B_{ux}^* &= R(u^*, t^*)F(u^{\neg *}, t^{\neg *}), \\ x^* &= R(u^*, B_{ux}^{\neg *})F(B_{ux}^*), \\ i^* &= R(B_{ux}^{\neg *}), \\ j^* &= R(B_{ux}^*), \end{aligned}$$

where B_{ux}^* has the same sign as x^*.

LEMMA 10. *Given a simply-driven STG with $u^* \in En(x^*)$, u in PI, and x in PO, applying TP for* Liable *model guarantees the underlying output signal x to be effective, stable, and precise with respect to input signal u.*

PROOF. This proof is similar to the proof of Lemma 5. □

4.4.2 Optimization

In optimizing TPs for *Liable* model, two possible cases are considered. If $u^* \in_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$, as the case of Or^+ and le^+ in T-element, we may simplify TP as

$$x^* = R(u^*)F(u^{\neg *}).$$

This optimization may impose restriction on system concurrency. An additional edge is added to the original STG, as the red dash line shows in Figure 5(b), in order to clearly specify the transition order between $u^{\neg *}$ and $x^{\neg *}$. The simplified TP guarantees signal x to be precise.

On the other hand, if $u^* \notin_\diamond \mathbf{P_s}(x^{\neg *}, x^*)$, an input signal t should be designated. Signal t is chosen such that $x^* \in En(t^*)$, $x^{\neg *} \in En(t^{\neg *})$, and $t^* \in_\diamond \mathbf{P_s}(x^*, u^{\neg *})$. An STG example is shown in Figure 5(c). Taking the STG of Figure 6(b) as another example, x^*, u^* and, t^* correspond to PO_1^+, PI_1^+, and PI_2^+, respectively. The TP in this case can be simplified as

$$x^* = R(u^*, t^{\neg *})F(u^{\neg *}, t^*).$$

The TP is simplified by adding an extra edge to the original STG, as the red dash line shows in Figure 5(c). System concurrency is reduced since transition $u^{\neg *}$ cannot occur until transition $t^{\neg *}$ completes. The simplified TP guarantees signal x to be precise.

LEMMA 11. *Given a simply-driven STG, the above optimization for the* Liable *model guarantees the modified STG to be deadlock-free and conservative to the original STG.*

PROOF. This proof is similar to that of Lemma 9. □

4.5 Independent

The TP for *Independent* model is complicated due to the fact that $u^{\neg *}$ is concurrent to both x^* and $x^{\neg *}$. Two internal signals B_{ux1} and B_{ux2} are added in order to keep signal x stable and avoid deadlock. *The fork from input signal u to output signal x, to internal signal B_{ux1} and to internal signal B_{ux2} is designated isochronic.* One input signal t and two output signals i and j need to be designated. We choose signals i and j with $i^* \in En(u^*)$ and $j^* \in En(u^{\neg *})$, and designate signal t satisfying $x^* \in En(t^*)$ and $x^{\neg *} \in En(t^{\neg *})$. An STG example is shown in Figure 4(e). Taking the STG of Figure 9(c) as another example, x^*, u^*, t^*, i^* and j^* correspond to PO_1^-, PI_2^+, PI_1^-, PO_2^+ and PO_2^-, respectively. The TP for *Independent* model is

$$\begin{aligned} B_{ux1}^* &= R(u^*, t^{\neg *})F(t^*), \\ B_{ux2}^* &= R(u^*, t^*)F(u^{\neg *}), \\ x^* &= R(B_{ux1}^*, B_{ux2}^{\neg *})F(B_{ux1}^{\neg *}, (B_{ux2}^* + u^{\neg *})), \\ i^* &= R(B_{ux2}^{\neg *}), \\ j^* &= R(B_{ux1}^*), \end{aligned}$$

where B_{ux1}^* and B_{ux2}^* has the same sign as x^*.

LEMMA 12. *Given a simply-driven STG with $u^* \in En(x^*)$, u in PI, and x in PO, applying TP for* Independent *model guarantees the underlying output signal x to be effective, stable, and precise with respect to input signal u.*

PROOF. This proof is similar to the proof of Lemma 5. □

5. CIRCUIT SYNTHESIS

Based on the behavior models and TPs introduced in Sections 3 and 4, we propose an algorithm to derive TC for every output signal of a simply-driven STG. Furthermore we design a circuit module called Up-Down-Hold module in order to implement QDI circuits from TCs.

5.1 Synthesis Algorithm

The algorithm *STGSynthesis* in Figure 7 firstly categorizes every enabling transition into the five behavior models for each output signal, and then TPs are applied based on the corresponding model. Optimization can be performed by applying optimizing TPs proposed in Section 4. The algorithm returns $R_b = \{R_b(x) | x \in PO\}$ and $F_b = \{F_b(x) | x \in PO\}$, which are the pull-up and pull-down sets for output signals.

In lines 3 to 6, function $ComputePs(G, a^*, b^*)$ computes the transitions in $\mathbf{P_s}(a^*, b^*)$, which are computable in time complexity $O(|E|)$ by traversing the STG G. The function *Classify* in line 7 categorizes the behavior model of x^* and its enabling transition u^*, as detailed in Section 3, based on the information calculated in lines 3 to 6. With the behavior model M_b we derive TCs for output signals according to TPs we derived in Section 4. If reducing concurrency is acceptable, optimization is applied in line 9; otherwise, direct synthesis is applied in line 11. Since the above computation is executed at most $|E|$ iterations, the overall complexity is $O(|E|^2)$.

THEOREM 2. *Given simply-driven STGs, algorithm STGSynthesis guarantees every output signal to be effective, stable, and precise if no optimization is applied.*

PROOF. The final TC for every output signal is derived by conjuncting all the necessary conditions through the algorithm. By Lemmas 5, 7, 8, 10 and 12, the output signals are guaranteed to be effective, stable, and precise. □

Algorithm *STGSynthesis*
Input: STG $G = (V, E)$
Output: Triggering conditions for output signals, R_b and F_b

```
for each x ∈ PO do
    for each u* ∈ En(x+) ∪ En(x−) do
        P_s^1 := ComputePs(G, x+, x−)
        P_s^2 := ComputePs(G, x+, x+)
        P_s^3 := ComputePs(G, x−, x+)
        P_s^4 := ComputePs(G, x−, x−)
        M_b := Classify(G, u*, P_s^1, P_s^2, P_s^3, P_s^4)
        if reducing concurrency is acceptable then
            (R_b, F_b, G) := Optimize(G, u*, M_b)
        else
            (R_b, F_b) := DirectSynthesis(G, u*, M_b)
return R_b and F_b
```

Figure 7: Algorithm: Derive corresponding behavior models and triggering conditions

THEOREM 3. *Given a simply-driven STG, algorithm STGSynthesis guarantees the underlying STG of the optimized circuit to be conservative to the original STG.*

PROOF. Applying TPs for *Staggered*, *Back-to-Back* and *Independent* models guarantees the circuits to be effective and stable according to Lemmas 5, 7 and 12, and therefore no alteration is made on the STG in such cases. As for *Sneaking* and *Liable* models, the modified STG is conservative due to Lemmas 9 and 11 if optimization is applied. Therefore by Definition 6 the final modified STG is conservative to the input STG. □

5.2 Circuit Implementation

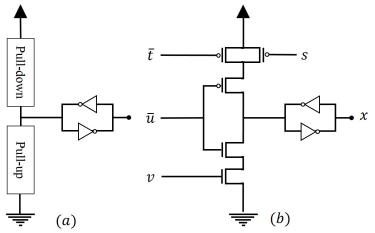

Figure 8: (a) UDH module, (b) circuit example.

The triggering conditions, $R_b(x)$ and $F_b(x)$, of a signal x can be directly translated into physical circuits. We propose a circuit module called Up-Down-Hold (UDH) module, which consists of a pull-up part, a pull-down part, and a state holding buffer as shown in Figure 8(a). The pull-up and pull-down sets of TCs can be directly implemented in the pull-up and pull-down parts, respectively, in a UDH module. Consider the TC example $R_b(x) = \{u^-, v^+\}$ and $F_b(x) = \{u^+, (t^+ + s^-)\}$, where u, v, t, s are input signals and x is the output signal to be implemented; the synthesized circuit is shown in Figure 8(b).

6. EXPERIMENTAL RESULTS

Our proposed synthesis procedure, named STG2QDI, were implemented in the C++ language. All experiments were conducted on a Linux machine with a Xeon 2.3 GHz CPU and 32 GB RAM. The benchmark STGs were partly obtained from [22], from translation of the write-back unit [24], and from translation of AMBA-AHP [25]. To assess the scalability and effectiveness of our proposed method, we further created four STG families (including `sequence-N`, `parallel-N`, `pipeline-A-N` and `pipeline-B-N`, which are parameterized on N) as shown in Figure 9. STG `sequence-N` specifies the protocol in which N processes work sequentially, with output signal PO_i and input signal PI_i signifying the permission and completion detection of the ith operation, respectively. Similarly, STG `parallel-N` specifies the protocol in which N processes work concurrently. STGs `pipeline-A-N` and `pipeline-B-N` were built to examine how STG structures affect circuit cost.

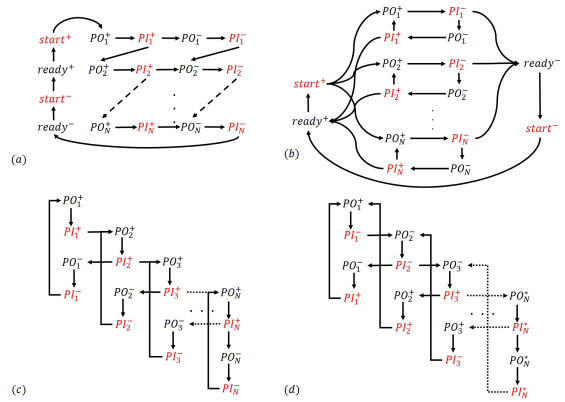

Figure 9: STGs for experiment: (a) sequence-N, (b) parallel-N, (c) pipeline-A-N, and (d) pipeline-B-N.

Since there exists no other synthesis tool that can synthesize QDI circuits from STG specifications, we only compared STG2QDI to two related, but not directly comparable, well-known asynchronous synthesis tools BALSA [26] and PETRIFY [9]. PETRIFY can synthesize speed independent (SI), but not the more robust QDI, circuits from STG specifications. Note that BALSA is designed for entire system synthesis rather than controller circuit synthesis and may not be an ideal tool for comparison.

The experimental results are shown in Table 1, where Columns 2, 3, and 4 show the numbers of input signals, output signals, and edges, respectively, of the corresponding STGs listed in Column 1, Columns 5, 7, 9 and 11 show the transistor counts of circuits synthesized by STG2QDI and by BALSA, either without or with applying the optimization techniques, Columns 6, 8, 10, 12 show the runtime (in seconds) of STG2QDI and BALSA, either without or with applying optimization techniques, Column 13 shows the transistor counts of circuits synthesized by PETRIFY, and Column 14 shows the runtime of PETRIFY with the dominating CSC solving time reported in the parentheses. Note that an entry of "-" in the table indicates data unavailable due to either the BDD memory overflow error (indicated as M.E.) of PETRIFY or timeout (indicated as T.O.) at 8000 seconds. Note that the transistors of inverters for input signals are not counted in STG2QDI and in PETRIFY whereas the transistor counts for BALSA are based on technology mapping with its own standard cell library. It should be emphasized that the three methods are not directly comparable due to their different synthesis assumptions. Nevertheless, to assess the optimality of STG2QDI the benchmark STG `t-element` [22] provides a good reference point: Its manual design [22] uses 14 transistors whereas STG2QDI uses 16 transistors, close to the manually optimized design.

Note that the STGs named "new" in [22] are modified from their counterparts named "conv." (conventional) to enhance system concurrency. Moreover, STGs `parallel-N` are more concurrent than STGs *sequence-N*. The experimental results reveal that circuits synthesized from STGs whose underlying systems behave more concurrently might require more transistors in direct implementation, but could be even smaller after optimization. This phenomenon is expectable since our optimization techniques simplify TPs by exploiting a system's concurrency as flexibility. Although our optimization techniques add extra edges to STGs, the gained reduction on circuit sizes can be significant. The tradeoff between circuit cost and system concurrency is an interesting issue for further study. In addition, the circuit cost difference between `pipeline-A-N` and `pipeline-B-N` suggests that STGs with more transitions be-

Table 1: STG synthesis results.

STG	Size			Stg2Qdi				Balsa				Petrify	
				w/o opt		w/ opt		w/o opt		w/ opt			
	#in	#out	#edge	count	T (s)	count	T (s)	count	T (s)	count	T (s)	count	T (s)
c-elem. [22]	2	1	8	8	0.01	8	0.01	248	0.51	248	0.53	10	0.01 (0.00)
s-elem. [22]	2	2	8	34	0.01	32	0.01	774	0.57	744	0.57	14	0.02 (0.01)
t-elem. [22]	2	2	9	36	0.01	14	0.01	806	0.56	414	0.59	12	0.03 (0.02)
concur-conv. [22]	3	3	13	62	0.01	59	0.01	1307	0.61	1261	0.61	32	0.14 (0.11)
concur-new [22]	3	3	15	66	0.01	23	0.01	1371	0.61	617	0.55	28	0.40 (0.36)
FV-conv. [22]	3	3	12	40	0.01	38	0.01	933	0.60	903	0.55	16	0.05 (0.04)
FV-new [22]	3	3	18	54	0.01	52	0.01	1207	0.60	1177	0.62	20	0.36 (0.32)
seq.-conv. [22]	3	3	12	40	0.01	38	0.01	933	0.57	903	0.58	16	0.05 (0.03)
seq.-new [22]	3	3	13	44	0.01	21	0.01	993	0.63	587	0.61	14	0.13 (0.11)
write-back-unit	7	7	38	200	0.01	156	0.01	3901	0.78	3163	0.69	152	150.81 (140.39)
AMBA-AHP	17	14	88	407	0.01	193	0.01	7926	1.07	4308	0.80	-	M.E.
sequence-5	6	6	24	102	0.01	96	0.01	2182	0.63	2098	0.60	82	9.37 (9.17)
sequence-10	11	11	44	187	0.01	176	0.01	3978	0.70	3824	0.72	174	170.50 (168.49)
sequence-40	41	41	164	697	0.01	656	0.01	14538	1.32	13964	0.34	-	M.E.
parallel-5	6	6	38	157	0.01	55	0.01	3096	0.72	1342	0.63	76	26.55 (25.39)
parallel-10	11	11	73	307	0.01	105	0.01	5947	0.88	2509	0.68	-	M.E.
parallel-40	41	41	283	1207	0.02	405	0.02	22921	1.94	9349	1.16	-	M.E.
pipeline-A-5	5	5	24	166	0.01	121	0.01	3197	0.72	2441	0.70	96	41.33 (40.79)
pipeline-A-10	10	10	49	366	0.01	266	0.01	6895	0.98	5228	0.85	-	T.O.
pipeline-A-40	40	40	199	1566	0.01	1136	0.01	28987	2.35	21788	1.73	-	M.E.
pipeline-B-5	5	5	24	127	0.01	38	0.01	2531	0.67	1017	0.56	52	2.57 (2.32)
pipeline-B-10	10	10	49	277	0.01	78	0.01	5390	0.88	2034	0.61	138	385.95 (337.58)
pipeline-B-40	40	40	199	1177	0.01	318	0.01	22402	1.75	7972	0.94	-	M.E.

longing to `Staggered`, `Sneaking`, and `Liable` models might require fewer transistors to implement.

For Balsa synthesis, we firstly derived the TCs for output signals of an STG through our algorithm and then translated the TCs into Balsa codes for QDI circuit synthesis. From the results of Table 1, we observe that the circuits synthesized by Stg2Qdi require much fewer transistors than those synthesized by Balsa. However, this comparison can be unfair due to the fact that the translation from TCs to Balsa codes is indirect and makes Balsa synthesis ineffective.

When Stg2Qdi and Petrify are compared, the transistor counts of Petrify are in general smaller with a few exceptions. However Petrify can be time consuming due to its CSC computation and, moreover, some cases cannot even be synthesized because of the state explosion issue.

7. CONCLUSION

We have analyzed the topological relations between transitions of a given valid and single-cycle STG, and defined five behavior models which completely cover all possible relations. We also have proposed the first synthesis algorithm that synthesizes simply-driven STGs into QDI circuits. Experimental results suggest that the our approach is effective. Case study on AMBA-AHB protocol also suggests the applicability in real-life applications.

REFERENCES

[1] A. Martin and M. Nystrom. Asynchronous techniques for system-on-chip design. *Proceedings of the IEEE*, 94(6):1089–1120, 2006.

[2] A. Kondratyev and K. Lwin. Design of asynchronous circuits using synchronous CAD tools. *IEEE Design & Test of Computers*, 19(4):107–117, 2002.

[3] A. Taubin, J. Cortadella, L. Lavagno, A. Kondratyev, and A. Peeters. *Design Automation of Real-Life Asynchronous Devices and Systems*, 2007.

[4] F.-C. Cheng, Y.-F. Chen, S.-C. Huang, and C.-Y. Huang. Synthesis of QDI FSMs from synchronous specifications. In *Proc. Int. Symp. on Asynchronous Circuits and Systems*, pp. 61–68, May 2014.

[5] C.-C. Chuang, Y.-H. Lai, and J.-H. R. Jiang. Synthesis of PCHB-WCHB Hybrid Quasi-Delay Insensitive Circuits. In *Proc. Design Automation Conf.*, 2014.

[6] T. Chu. Synthesis of self-timed VLSI circuits from graph-theoretic specifications. Technical report, MIT, 1987.

[7] S. Golubcovs, W. Vogler, and N. Kluge. STG-based resynthesis for Balsa circuits. In *Proc. Int. Conf. on Application of Concurrency to System Design*, pp. 140–149, 2013.

[8] J. Gu and R. Puri. Asynchronous circuit synthesis with Boolean satisfiability. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, 14(8):961–973, 1995.

[9] J. Cortadella, M. Kishinevsky, A. Kondratyev, L. Lavango, and A. Yokovlev. Petrify: A tool for manipulating concurrent specifications and synthesis of asynchronous controllers. *IEICE Trans. on Information and Systems*, E80-D(3):315–325, 1997.

[10] D. Wist, M. Schafer, W. Vogler, and R. Wollowski. STG decomposition: Internal communication for SI implementability. In *Proc. Int. Conf. on Application of Concurrency to System Design*, pp. 13–23, 2010.

[11] V. Khomenko. Logic decomposition of asynchronous circuits using STG unfoldings. In *Proc. Int. Symp. on Asynchronous Circuits and Systems*, pp. 3–12, 2011.

[12] J. Carmona, J. Colom, J. Cortadella, and F. Garcia-Valles. Synthesis of asynchronous controllers using integer linear programming. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, 25(9):1637–1651, 2006.

[13] V. Khomenko, M. Koutny, and A. Yakovlev. Logic synthesis for asynchronous circuits based on Petri net unfoldings and incremental SAT. In *Proc. Int. Conf. on Application of Concurrency to System Design*, pp. 16–25, 2004.

[14] S.-B. Park and T. Nanya. Automatic synthesis of speed-independent circuits from signal transition graph specifications. In *Proc. Int. Conf. on VLSI Design*, pp. 389–392, 1996.

[15] S.-T. Jung, E.-S. Park, J.-S. Kim, and C.-S. Jhon. Automatic synthesis of gate-level speed-independent control circuits from signal transition graphs. In *Proc. Int. Symp. on Circuits and Systems*, pp. 1211–1214, 1995.

[16] C. Ykman-Couvreur, B. Lin, G. Goossens, and H. De Man. Synthesis and optimization of asynchronous controllers based on extended lock graph theory. In *Proc. European Conf. on Design Automation*, pp. 512–517, 1993.

[17] E. Pastor and J. Cortadella. Polynomial algorithms for the synthesis of hazard-free circuits from signal transition graphs. In *Proc. Int. Conf. on Computer-Aided Design*, pp. 250–254, 1993.

[18] R. Puri and J. Gu. Signal transition graph constraints for speed-independent circuit synthesis. In *Proc. Int. Symp. on Circuits and Systems*, pp. 1686–1689, 1993.

[19] S. Sudeng and A. Thongtak. Signal transition graph based logic synthesis for asynchronous control circuits using template based method. In *Proc. IEEE Region 10 Conf.*, pp. 1–4, 2007.

[20] R. Fuhrer, S. Nowick, M. Theobald, N. Jha, B. Lin, and L. Plana. Minimalist: An environment for the synthesis, verification and testability of burst-mode asynchronous machines. Technical report, Columbia University, 1999.

[21] T. Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541–580, 1989.

[22] L. Plana, S. Taylor, and D. Edwards. Attacking control overhead to improve synthesised asynchronous circuit performance. In *Proc. Int. Conf. on Computer Design*, pp. 703–710, 2005.

[23] J. Sparso and S. Furber. *Principles of Asynchronous Circuit Design: A Systems Perspective*. Springer, 2001.

[24] R. Manohar, T.-K. Lee, and A. Martin. Projection: A synthesis technique for concurrent systems. In *Proc. Int. Symp. on Asynchronous Circuits and Systems*, pp. 125–134, 1999.

[25] AMBA Specifications. `http://www.arm.com/products/system-ip/amba/`.

[26] The Balsa Asynchronous Circuit Synthesis System. `http://apt.cs.manchester.ac.uk/projects/tools/balsa/`.